

Neural Field Write Up

Jonathan Grothe

September 15, 2026

1 Introduction

The intent of this document is to compile what I’ve learned about the neural field model proposed in Allocentric Flocking (Salahshour and Couzin (2025)). I began by exploring the architecture of the model, then worked towards recreating the experimental trajectories seen in The Geometry of Decision Making (Sridhar et al. (2021)), and ended my analysis by exploring how the model performs in a more general situation.

2 Model Basics

This model uses ring attractor dynamics to turn external signals into movement in a neurobiologically feasible way. The model is set up as a ring of 100¹ neurons that can receive external input and that are connected to each other neuron in the ring through a connectivity profile of short range excitation and long range inhibition.

My analysis focuses primarily on how the configurations of the targets influence the movement produced by the ring attractor. Each target is represented as a point with an attraction (h_0) and a width (σ), which gives it a shape like a gaussian. In most cases when there were several targets I assumed that they all had equal h_0 and σ , however in section 3.3 I explored what happened when one target had a greater h_0 than the other targets. The activity received by a neuron can be scaled by the distance from the agent to the source of the external input using equation 1 (referred to as distance dependence). In this equation d_{AB} is the distance from the agent to the target and L is the total length and width of the arena.

$$a_t = h_{0t} * \exp - \frac{\text{adistf} * d_{AB}}{L} \quad (1)$$

Equation 2 dictates how much input a neuron n receives from a target t . Each neuron will receive input from each target, however, as the difference in the angle the neuron encodes and the angle of the target relative to the agent’s current position (Δang) increases, the

¹This is a parameter, and changing it will be discussed in section 4, however unless explicitly stated I used 100 neurons.

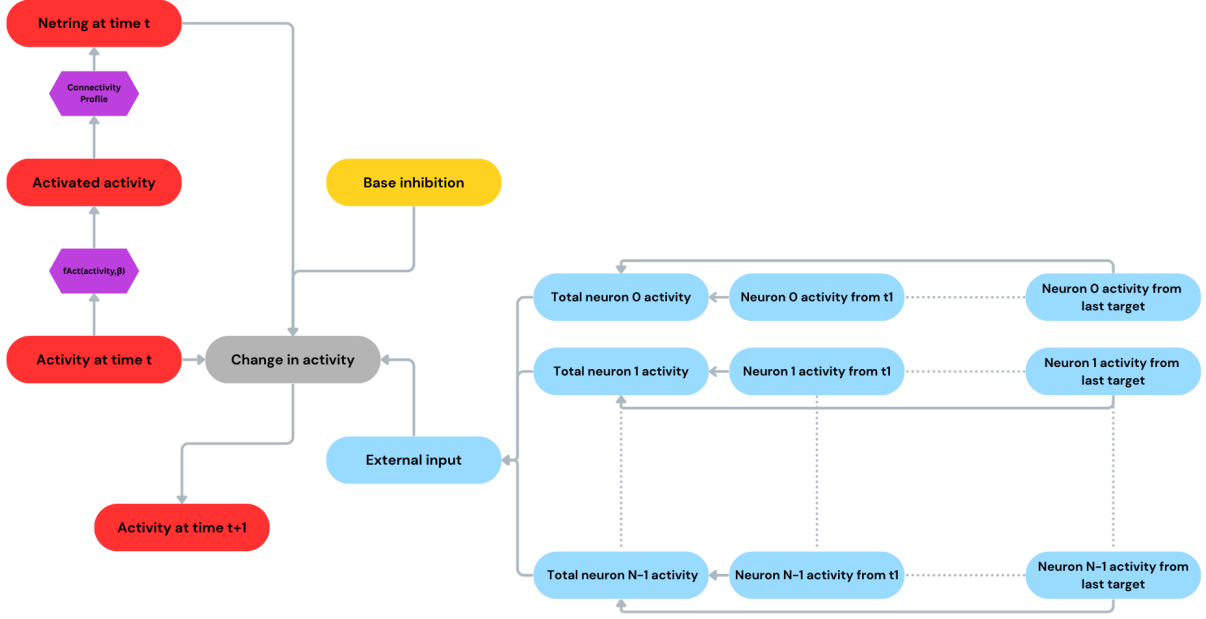


Figure 1: The architecture of the neural field model.

external input to that target decreases.

$$\text{ext}_{tn} = a_t * \exp \frac{-0.5 * \Delta \text{ang}^2}{\sigma^2} \quad (2)$$

Within the ring attractor, there is an inverse neural noise parameter, β , that brings positive activity close to 1 and negative activity close to 0 using the function shown in equation 3. β does not represent random noise, but rather the ability (or lack thereof) to receive accurate signal from the targets.

$$u_{\text{act}} = \frac{1 + \tanh(\beta * u)}{2} \quad (3)$$

The influence of the external input is added to the previous state of the ring attractor, the connectivity profile of the previous state, and the constant inhibition to produce the new activity state, which is aggregated into a direction and magnitude. A flowchart of the layers of the model is shown in figure 1.

The ring attractor can encode direction egocentrically, which moves the entire ring attractor so that neuron 0 is at the current heading at each step, or allocentrically, which holds neuron 0 at 0 radians regardless of the heading direction. There is a threshold to switch from an allocentric encoding to an egocentric encoding in the code. I did not pursue using it, although I think it likely will be useful in further uses of this model.

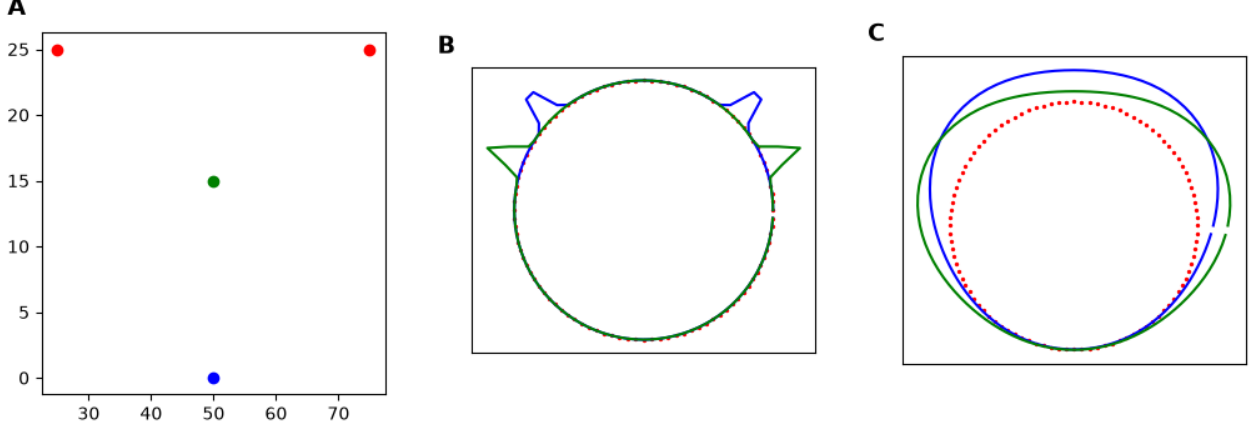


Figure 2: **A** shows the position of the agent at two points relative to two targets (in red). **B** shows the external input received at each position with $\sigma = 0.05$ and **C** shows the external input received at each position with $\sigma = 0.6$. Note the different shapes of the external input in **B**, which can be attributed to the angles to each target falling in between two neurons at the blue point, whereas at the green point the angles to each target falls very close to a neuron.

Encoding direction egocentrically introduces instability when the heading is changing. If the heading changes enough that neurons that had low activity at the previous time step are receiving the strongest external input, then the activity will be spread out, and after the base inhibition and activation function this will often lead to a nearly flat distribution. The consequences of this will be discussed more in section 3.4.

The rate of neural change and speed of the target are also parameters, however, I did not change these parameters throughout my systematic analysis. In the small amount of experimentation with these parameters I found that they could have some impacts on model performance, but their impact seemed smaller than the parameters I was interested in. In the future, it would be valuable to take another look at these parameters if changing them would improve the fidelity of the model.

When the targets are static (as is assumed throughout the remainder of this analysis), the external input will change each time the agent moves. As the agent moves, the smallest Δang in the ring attractor will change, as the angles to the targets will not be perfectly aligned with the directions that the neurons encode. This means that despite the targets staying exactly the same, how the agent perceives them will change slightly as the agent moves. This is especially relevant at low σ values. For example, if σ is 0.1 and there are 10 neurons (equally spaced around the ring at $\frac{\pi}{5}$ radians apart), if there is a target at $\frac{\pi}{10}$ radians, then the neurons at 0 radians and $\frac{\pi}{5}$ radians ($\Delta\text{ang} = \frac{\pi}{10}$) will receive approximately 0.00719 times the original attraction. If the target was at $\frac{\pi}{5}$ radians ($\Delta\text{ang} = 0$), then the neuron at $\frac{\pi}{5}$ would receive the original attraction and the neuron at 0 radians would receive approximately 2.68e-9 of the original attraction. With 100 neurons this phenomenon still occurs, but because the gaps between neurons are much smaller it is much less pronounced. An example of the different shapes of external input is shown in **B** in figure 2.

The only randomness is introduced in the initialization.² This means that any differences between two simulations ran with the exact same parameters can be traced back to the random initialization.

3 Parameter Spaces and Edge Cases

The first situations I worked towards recreating with this model were the trajectories seen in the experiments in Sridhar et al. (2021). The theoretical idea of a bifurcation is that, if we assume there are two targets which are equally far from the agent, the agent will begin by moving in the aggregated direction between the two targets. As it moves towards both targets, the difference in the angles for the agent to each target will increase, and because it has a limited field of vision, at a certain point it will have to choose one and proceed towards that target.

This video shows the layers of the ring attractor as it goes through the process of a bifurcation using an allocentric coordinate system. The bifurcation happens from approximately time 490 to time 600. This video shows a similar situation using an egocentric coordinate system. Here the entire reference frame shifts the same direction back and forth as the heading changes slightly in the aggregation phase, and then during the bifurcation the external input changes rapidly as the heading changes rapidly. These videos stopped when the agent reached a target, but the agent will continue to move towards the other target after reaching one. This video shows what happens when we let the simulation continue running in the allocentric case.

Once I established that this model could recreate this pattern with a variety of different parameters, I could sweep the parameter space to get a better idea of what parameters worked best for this situation.

3.1 Parameter Sweeps

To begin my analysis, I set $\beta = 100$ with the goal of simulating relatively low noise neural activity.

I found that for two equally spaced targets, there was generally a region of h_0 and σ values where the agent reached a target and the agent failed³ to reach a target outside that region. Figure 3 shows this over 100 attraction values between 0.15 and 0.4 and 100 σ values between 0.1 and 1.⁴ It appears that an h_0 around the base inhibition (0.2) is needed to provide

²For most of my analysis I randomly initialized the activity, however while studying the model in the future it might make more sense to seed the initial activity to guarantee deterministic behavior

³Failure is defined as not reaching the target within the time limit of 5000 time steps.

⁴Note that this figure was produced by initializing the ring attractor with zeros, which has the benefit of

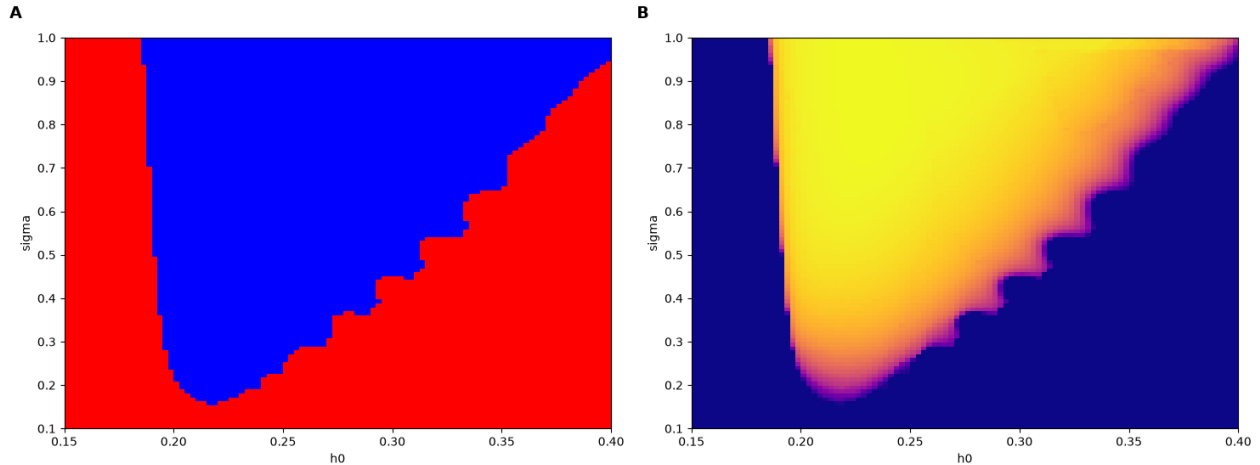


Figure 3: Sweeps across 100 attraction values between $[0.15, 0.15]$ and $[0.4, 0.4]$ (x axis) and 100 σ values between 0.1 and 1 (y axis). In **A** blue represents simulations where the agent reached a target and red represents simulations where the agent did not reach a target within 5000 time steps. **B** shows the time to reach the target, with lighter being faster. Each simulation was run one time with an initial activity of all zeros, no distance dependence, and an allocentric encoding.

enough activity to reach the target, but increasing h_0 will lead to failure at lower σ values. In these cases, I found that failure generally only occurred when there was too much input and the agent failed to turn towards a target or too little input and the agent moved too slowly to reach the target within the time limit.

The impact of σ on the trajectory in this geometry can be seen in figure 4. With the widest σ s, the input from the targets overlaps and takes longer to split, leading to a later bifurcation. With the narrowest σ s, the input from the targets remains low throughout most of the ring attractor, which means it takes longer for the inhibition of one bump to impact the activity of the other bump. Intuitively, it seems as though the σ at which the agent is the ‘best’ at deciding between two targets is somewhere in between.

In the two target case, the agent must deviate from the aggregated trajectory to reach a target, which means simply reaching a target is a good proxy for success. If we add another target in the same way as was done in Sridhar et al. (2021), this is not the case, as the agent can reach the middle target by simply continuing on the aggregated path. To recreate the experimental trajectories seen in Sridhar et al. (2021), the agent would have to be able to reach all three targets with the same settings and would have to bifurcate most of the time. Based on a few initial simulations, it seemed as though the agent would be heavily biased towards the middle target, so I began by sweeping to see if the agent could reach one of the two outer targets.

not needing to run multiple samples for each setting and thus drastically decreases runtime, but does have some bias.

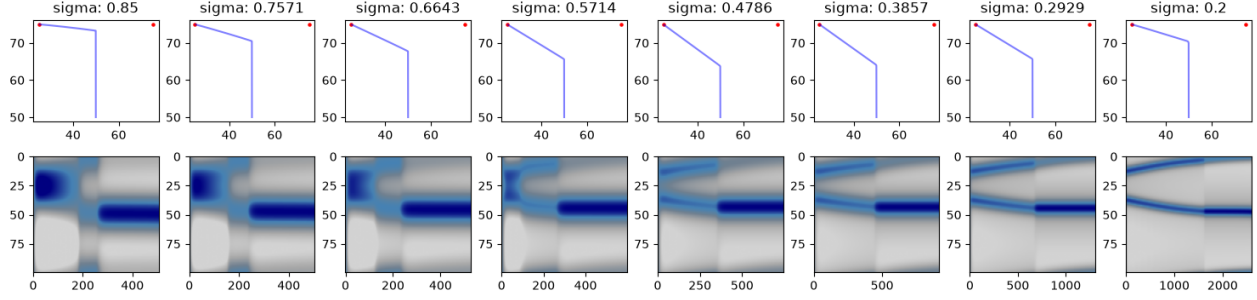


Figure 4: Trajectories and activity heatmaps for different values of σ . In the heatmaps the y axis is the activity of that neuron (with neuron 0 corresponding to 0 radians, neuron 25 corresponding to $\frac{\pi}{2}$ radians, etc.). The x axis is time. Darker blue corresponds to more activity. σ decreases from left to right but all the other parameters stay the same. Each simulation was run one time with an initial activity of all zeros, no distance dependence, and an allocentric encoding. Both targets have $h_0=0.23$

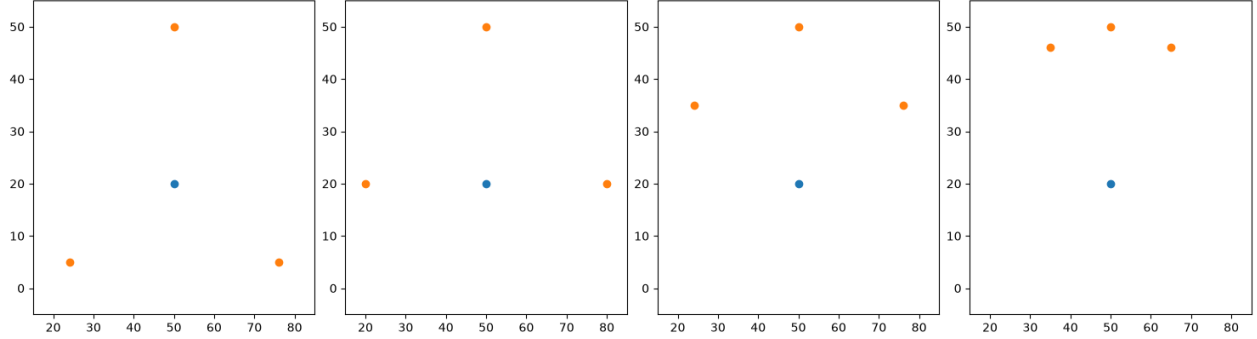


Figure 5: The setups I analyzed in the three target case, the targets are all 30 units from the starting point, the angles between the targets from left to right are $\frac{2\pi}{3}, \frac{\pi}{2}, \frac{\pi}{3}, \frac{\pi}{6}$.

I analyzed the three target case in four parts, keeping bilateral symmetry as I moved the targets closer to each other. The geometries I analyzed are shown in figure 5. The results of the bilateral symmetry sweeps (aside from the $\frac{2\pi}{3}$ radians case, which in the vast majority of cases the agent takes a straight line to one of the three targets) can be seen in figure 6. The severe drop off in reaching the outer target as the angles between the targets gets closer makes some sense, as when the targets are closer to each other on the ring the aggregated path becomes stronger, and therefore the sway of the outer targets will be small relative to the path the agent is already on near the ‘decision’ point.

The sweeps I performed here gave me a starting point for the rest of my analysis by proving that, within a set range of parameters, the model will reliably, quickly, reach a target. I hoped to get more information out of each simulation beyond reaching or failing to reach a target and the time taken to reach a target, and I tinkered with metrics to quantify the state of the neural activity and the trajectory (detailed in section 3.2). I think it may be possible to dive deeper into specific metrics for a sweep in this way, but in my analysis, I failed to find anything else to measure that I found meaningful.

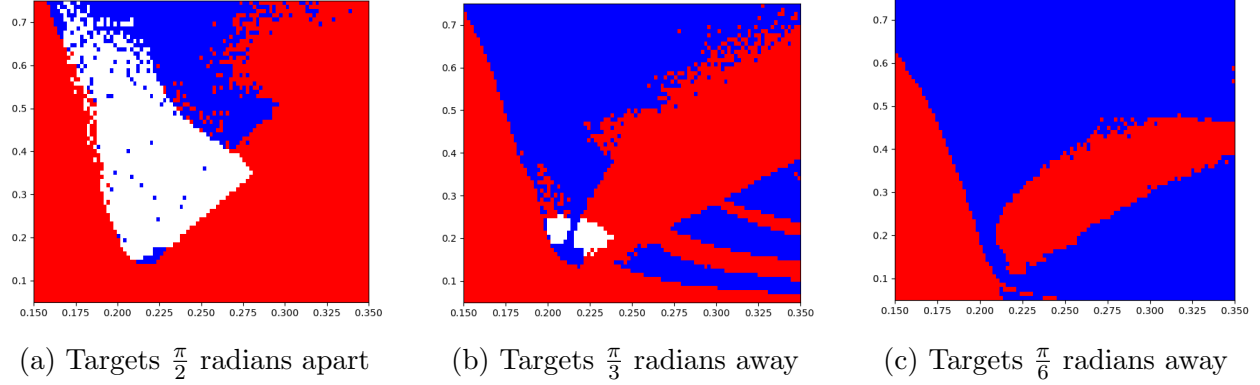


Figure 6: Sweeps over 101 attraction values from 0.15 to 0.35 and 71 σ values from 0.05 to 0.75, white represents reaching one of the two outer targets, blue represents reaching the center target, and red represents reaching no targets. Each neuron is initialized using a randomly generated number from a normal distribution multiplied by 0.2. The simulations used an allocentric encoding and no distance dependence.

3.2 Bifurcation Angle

It was one of my goals throughout this project to be able to find a simple rule of thumb for the angle at which the agent turns in a bifurcation as a function of parameters. To do this I needed to find a way to measure bifurcation angle from the trajectory. In the two target case, I approached this by attempting to find a bifurcation point, and defining the pre-bifurcation trajectory as the line from the starting position to the bifurcation point, and the post-bifurcation trajectory as the line from the bifurcation point to the ending position. From there I could measure the angle between the two lines.

Finding a bifurcation point that visually tracked with where the bifurcation appeared to occur was challenging due to the oscillation that occurs in the aggregation phase. This oscillation is visible in the change layer in the first ten seconds of the video of an allocentric bifurcation. Because on a small scale the bifurcation is a gradual turning motion, rather than abrupt change of direction, I tried splitting the bifurcation point into a start point and end point. Then I would measure the angle between the line connecting the start point and the start of the bifurcation and the line connecting the end of the bifurcation to the end point. I still struggled to accurately classify the start and end of the bifurcation. The errors accumulated when I tried to apply my measurement to different parameter spaces. Ultimately, the changes of the magnitude of the oscillation before the bifurcation point across different parameter spaces and random initializations led to enough falsely flagged bifurcation points that I gave up without a satisfactory way to get the bifurcation point from a trajectory.

The three target case was even more challenging because if I flagged one of the start or end points incorrectly, that error would then propagate to the next measurement. The agent also tended to turn more gradually in the three target cases, making the measurement of an angle less reasonable. This is seen in figure 7. While I thought being able to approximate bifurcation angle from the parameters could provide clues to general model behavior, I think

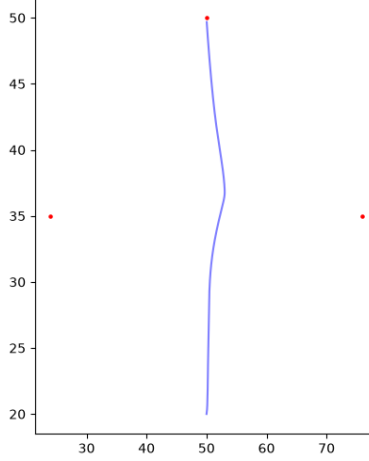


Figure 7: An example three target trajectory where the agent changes direction gradually. Targets are $\frac{\pi}{3}$ radians apart, 30 units from the agent, and all have $h_0 = 0.25$ and $\sigma = 0.5$. Coordinates are encoded allocentrically and there is no distance dependence.

the assumption that the agent is traveling on a straight line before and after the bifurcation is unreasonable in a more general scenario. The follow up to this analysis can be seen in my discussion of vector fields in section 4.

3.3 Unevenness

After looking at situations where the input from the targets was identical, the next step was to see how the model performed when one of the targets was more attractive than the other(s). In the three target case, there is some bias towards the middle target, as discussed in the prior section, so I looked at what happens when one of the outer targets is more attractive. It is also possible to introduce unevenness through the geometry while keeping the h_0 and σ of all the targets equivalent, which will be discussed in section 4.

The goals of this analysis were to prove that the model can detect differences while providing some context for what scale this detection takes place on. In figure 8 we see that, depending on the scale of the random initialization, the ability to detect differences changes. As the scale of the initialization increases, it takes more of a difference in attractiveness for the agent to reliably reach the better target. The level of detectable difference is dependent on parameters (such as base h_0), but this provides a good approximation of the scale of the difference that can be detected in relation to the initialization.

In the three target situation reaching the target is not so straightforward. Figure 9 shows that, when there is no distance dependence, as we increase the attraction of one of the outer targets there is a large region where there is one better target but the agent fails to reach any target. This video shows that the model can ‘detect’ the best target, and will move towards it early on, however, it doesn’t reach it. It appears that the conflict between the more attractive target and the inherent bias in the geometry pulling the agent towards the

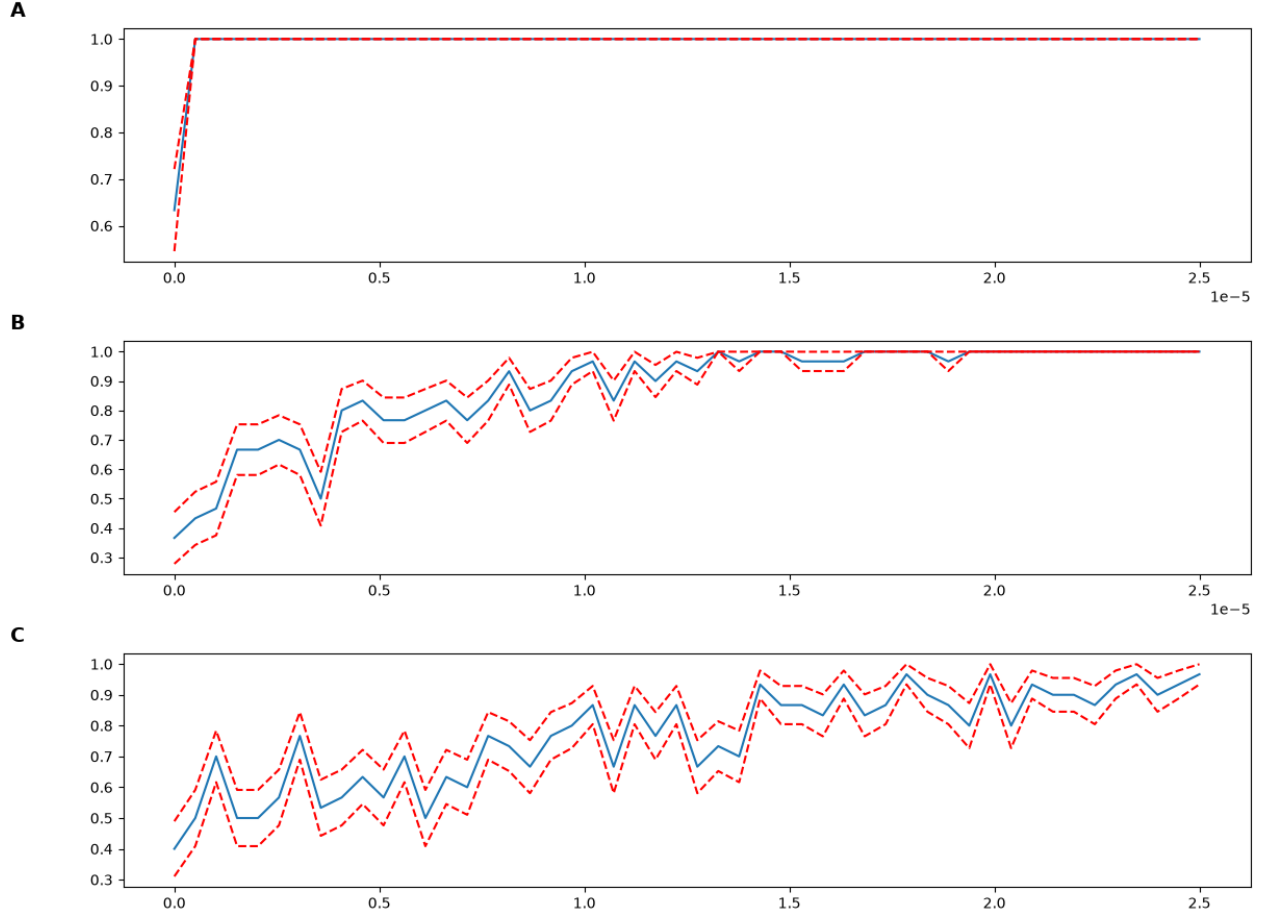


Figure 8: Each plot shows the probability of reaching the more attractive target in a simulation with targets at (25,75) and (75,75) with the agent starting at (50,20). The x axis is how much more attractive the target at (75,75) is than the target at (25,75). The blue line is the probability of reaching the more attractive target and the red dotted lines are the standard error of that probability. Each neuron’s initial state is determined by a number generated from a standard normal distribution multiplied by f . In **A** $f=0.001$, in **B** $f=0.2$, and in **C** $f=2$. The left and middle target have $h_0 = 0.25$, the right target has $h_0 = 0.25 + x$. The simulations were run with $\sigma=0.4$, $\beta=100$, no distance dependence, and an allocentric encoding.

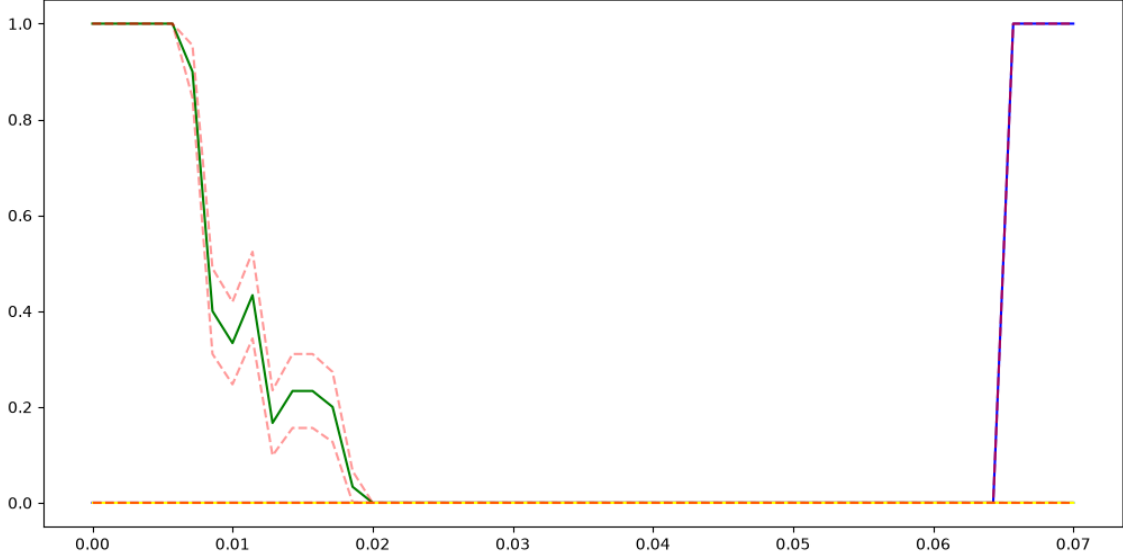


Figure 9: A sweep of probability of reaching each target as the attraction of the rightmost target increases. The middle target is green, the leftmost target is yellow, and the rightmost target is blue. Standard errors are shown around the probabilities in the red dashed lines. The left and middle target have $h_0 = 0.21$, the right target has $h_0 = 0.21 + x$. Each neuron is initialized using a randomly generated number from a normal distribution multiplied by 0.2. For each x , the probability of reaching each target is found by taking the proportion of times the agent reaches that target over 30 different random initializations. The simulations are run without distance dependence, using an allocentric encoding of direction, and with $\sigma = 0.4$

center target causes the agent to be caught in a repeated cycle of motion.

Figure 10 shows what happens when there is distance dependence. This video shows the layers of the neural network in the distant dependent case. Including distance dependence prevents the repeated motion, and the agent reaching the better target is inversely related to the agent reaching the middle target, as would intuitively make sense.

3.4 Egocentric

Throughout this document I have shown examples of allocentric encodings. Generally egocentric encodings lead to pretty similar results; however, the instability of shifting the ring attractor led to some hard to explain behavior, so I ended up not pursuing egocentric encodings much beyond the two target case.

In the previously mentioned two target scenario with one target more attractive to the others, when using an egocentric encoding there were parameter spaces where the agent would almost exclusively go to the less attractive target. This is seen on a large scale and small scale near the decision point in figure 11, as well as in this video. It seems as though the

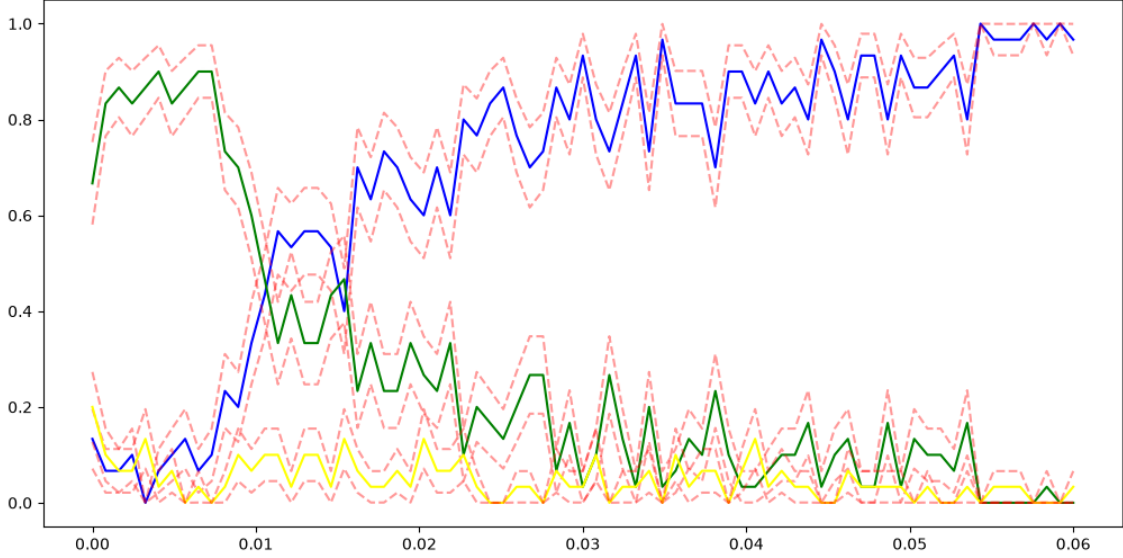


Figure 10: A sweep of probability of reaching each target as the attraction of the right-most target increases with distance dependence. The settings are the same as figure 9 except that the left and middle target have $h_0 = 0.25$, the right target has $h_0 = 0.25 + x$, and the simulation is run with distance dependence.

heading adjustment caused the agent to turn past the target that was more attractive, so, despite correctly moving towards that target initially, it consistently failed to actually reach that target. Furthermore, the drastically changing scales of movement near the decision point made this very tricky to study further.

I do think that the idea of a switch to egocentric near to a target to ensure that the agent actually succeeds in visiting a target could improve model performance, although I did not investigate this in depth. I will discuss my reasoning more in section 5.

4 Generalization

The next step was seeing how the patterns that had emerged in two and three target cases generalized to any given number of targets, arranged in any given way. For this I generated 10 targets with random positions a certain distance from the agent.

Before we examine this scenario, it is important to note that because the geometry is randomly generated there is a less clear way to evaluate if the agent is doing what we want it to. In this case I was looking for trajectories that more or less made sense. This primarily meant I wanted to avoid trajectories where the agent stopped moving relatively far away from any targets or trajectories where the agent moved in the same pattern repeatedly. Ideally, the agent would move in an aggregated direction for some of the simulation, but as it got closer to the targets it would change direction and reach a target.

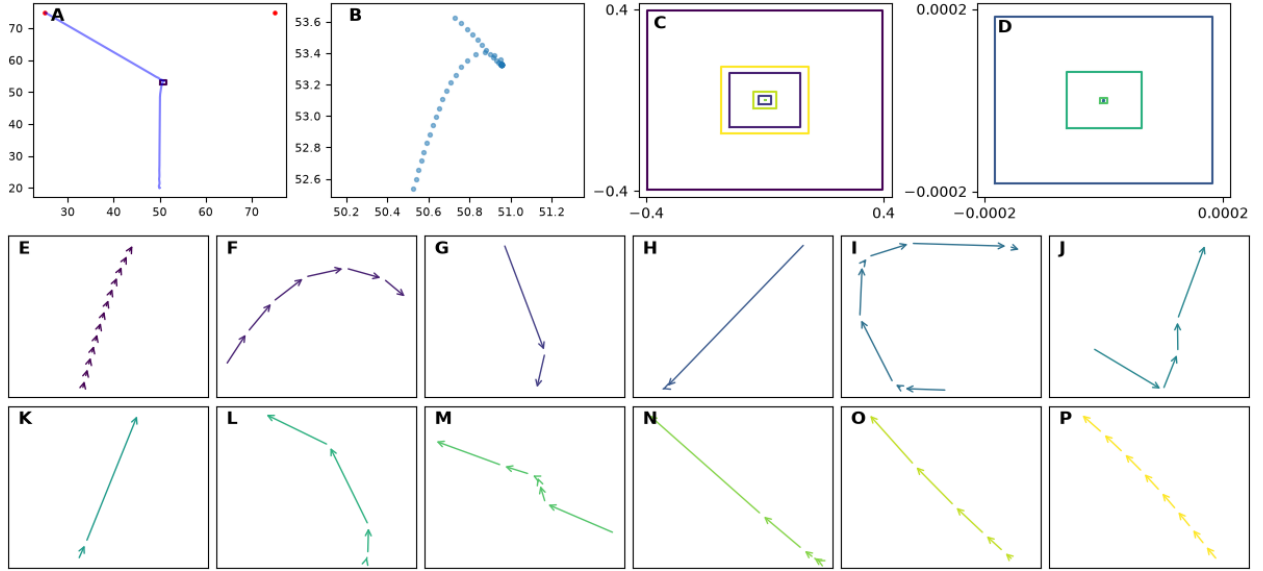


Figure 11: A zoomed in view of the agent's trajectory in a simulation where the agent goes to a less attractive target using an egocentric representation. **A** shows the overall trajectory of the agent, The agent starts at (50,50) and the targets are placed at (25,75) and (75,75), with h_0 of 0.25 and 0.255, respectively. The simulation is run with $\sigma = 0.4$, no distance dependence, $\beta = 100$, and each neuron was initialized from a normal distribution seeded with numpy global seed 24 multiplied by 0.2. **B** shows a zoomed in part of the trajectory (purple box in **A**). Figures **E** through **P** show smaller portions of this trajectory, at different scales. The scales are visualized in **C** (colors correspond to figures **E**, **F**, **G**, **N**, **O**, and **P**) and **D** (colors correspond to **H**, **I**, **J**, **K**, **L**, and **M**). Note the difference in the axis scales between **C** and **D**.

With more targets the external input has more potential to change with each movement from the agent. This increases the probability of repeated motion as seen in this video of the three target uneven attraction scenario. An example of the agent stalling out when there is no distance dependence is seen in this video. However, adding distance dependence does not necessarily solve this problem, as seen in this video, where the agent still gets stuck in repeated motion. When we increased adistf (equation 1) to 3, we saw that the agent does reach a target, as demonstrated in this video. Strong distance dependence seems to more often lead the agent to a target, but from what I have seen, as the number of targets increases the probability of becoming trapped in repeated motion increases.

Figure 12 compares the probability of reaching a target using distance dependence and not using distance dependence over 1000 samples. We can see that distance dependence leads to slight improvements in the probability of reaching the target. However, the agent still fails to reach the target a large proportion of the time, and there are a fair amount of scenarios where distance dependence fails and no distance dependence is successful (purple section in the rightmost bar). This suggests that using this distance dependence function should be considered given the geometry, rather than adopting a general framework and applying it to an arbitrary geometry. This also suggests that the current distance dependence function could be tweaked to produce better results, as with how the function is currently written, increasing distance dependence runs the strong risk of shrinking the external input so much that the agent moves very slowly or not at all.

From there I worked to get an understanding of what characteristics of a geometry to look out for when thinking about distance dependence. My first pass at this was to create a vector field that defined the external input at each position in the grid. Figure 13 shows this in **E**. Each arrow found in **E** in figure 13 is the angle from the agent to the average position of the targets. In the future, this could be adjusted by accounting for different h_0 values or other parameters. The external input vector field approximates the trajectory well early on, but as the bump splits, the actual trajectory veers away from the approximated trajectory.

From our previous analysis about σ and aggregated versus separate bumps (as seen in figure 4), we know that σ will impact when this split occurs. This may be an interesting analysis to follow up on, but with more targets, I found tracking targets to bumps in the ring attractor to be challenging (note the different widths and magnitudes of bumps in **C** in figure 13).

I briefly considered the possibility of decreasing σ so that more often each target could create a narrow bump that was distinct from other bumps. This may be worth pursuing in some effect, however, as mentioned in section 2, with a low σ , there will be more information lost when the neurons misaligned with the targets. This means that to support a lower σ while avoiding losing signal we need to increase the number of neurons, which significantly increases run time.

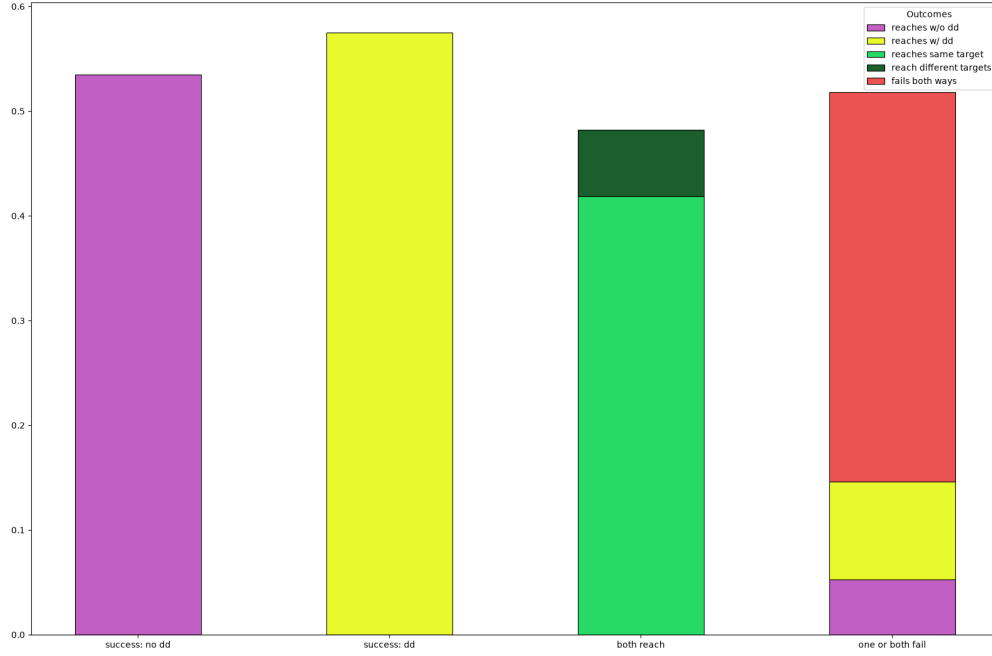


Figure 12: Probability of reaching the target, 1000 simulations of 10 randomly generated points that are at least 5 units away from each other, for each set of points the simulation was run once with distance dependence and once without. The agent started 50 units down from the center of where points were generated. The farthest a point could be from the center was 20 units. From left, the bars show: the probability of reaching a target without using distance dependence, the probability of reaching a target while using distance dependence, the probability that both reach the same target (light green) and the probability that distance dependence and non distance dependence reach different targets (dark green), the probability of reaching a target without distance dependence when distance dependence doesn't reach a target (purple), the probability of reaching a target with distance dependence when no distance dependence doesn't reach a target (yellow), and the probability that neither reaches a target (red).

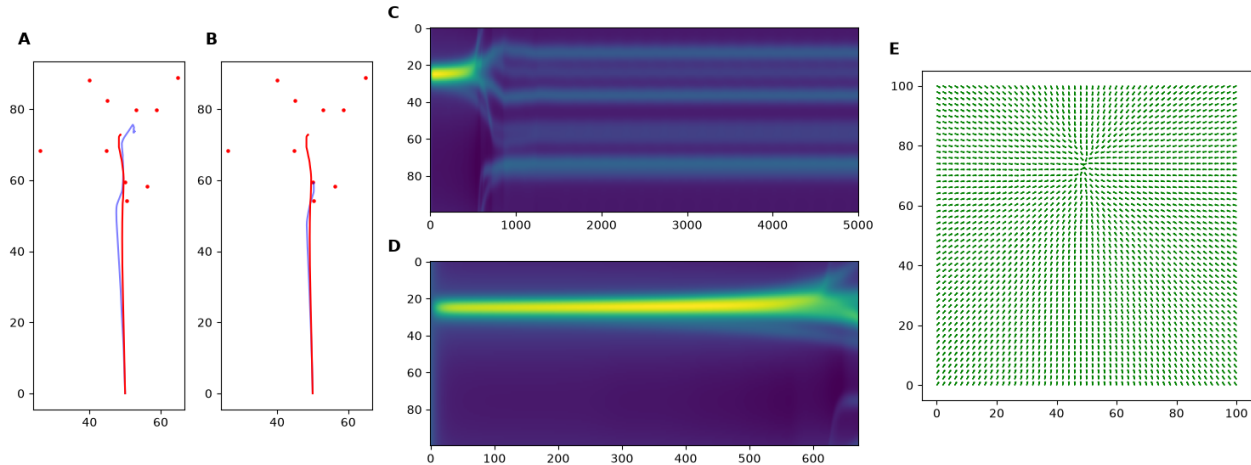


Figure 13: **A** (no distance dependence) and **B** (distance dependence) show the trajectories (in blue) for a simulation with randomly generated points that are at least 5 units apart. The red line is the trajectory if the agent followed the vector field of external input (**E**). **C** shows the neuron heatmap for **A** and **D** shows the neuron heatmap for **B**.

5 Conclusion and Further Steps

This purpose of this document is to describe a few relatively unstructured explorations of this neural field model to help guide a more structured use of it in the future. Hopefully, when provided with an arbitrary animal trajectory that could reasonably be modeled this way, this document provides a starting point for which parameters should be tuned to get closer to that trajectory.

The first explorations I did with this model were focused on the simple two target case discussed in section 3. This was interesting and helpful, but I found it hard to apply anything I learned in a higher parameter space, because the idea of a decision point completely changes as the agent is pulled in more directions.

As I explored the egocentric case and specifically the curiously deterministic behavior of moving towards a less attractive target 3.4, I found hugely different timescales near the ‘decision point.’ Near a decision point it makes sense to move slower in order to pick up on smaller differences. However, using an egocentric encoding of direction in the current configuration of the model somewhat defeats this, as the heading will update relative only to the direction of the input, not the magnitude, which will then have cascading effects throughout the ring attractor. In this video at around the 6 second mark the destabilizing effects of this can be seen throughout all layers of the ring attractor.

If this or anything similar causes problems in the future, it may be valuable to consider letting several updates occur in the ring attractor for each movement step. This would increase the run time and would make the distance traveled at each step more uniform (losing the slow movement near the decision). It is also worth considering further if this would be more accurate to the way the brains of the animal of interest work.

When generalizing to an arbitrary geometry, I found it challenging to land on a clear strategy using the parameters to increase the likelihood that the model produces a ‘good’ trajectory. It seems as though when the agent is in the middle of the targets and is most sensitive, if the external input changes enough when the agent moves, it will fail to reach a target. In the future I think finding how much change in external input is tolerated without getting stuck in a cyclical motion would be interesting.

If I was to change the way the model is constructed after my analysis, I would be most curious about a different distance dependence function. I did not explore it in depth, but I also think a change to how the egocentric switch occurs would be interesting. I think toying with the idea of an egocentric switch based on the angle to each target rather than the distance to the nearest target could be interesting, as this seems to play a role in determining when the agent gets stuck in repeated motion.

Random noise has very little impact on this model. For most of my analysis I did not fully grasp the importance of this, and I left this analysis wishing I had paid more attention to the initialization as a parameter. Relatedly, it seems to me that 3 is a reasonable way to introduce neural noise, although I think adding more random noise would be interesting and potentially valuable as a way to allow the model to produce more trajectories within a given parameter space.

How true this model is to actual neurobiological principles is certainly out of my depth, however, if there is a neurobiologically feasible alternative to the currently implemented connectivity profile it would be interesting to see how that impacts the performance of the model. Similarly, if the idea of running through several neural steps before moving makes neurobiological sense I would encourage implementing that in the future.

The repository with the code I used to run all the simulations is included here. The YouTube channel with all the videos included in this report is linked here.

References

- Salahshour, M. and Couzin, I. D. (2025). Allocentric flocking. *Nature Communications*, 16(9051).
- Sridhar, V. H., Li, L., Gorbonos, D., Nagy, M., Schell, B. R., Sorochnik, T., Gov, N. S., and Couzin, I. D. (2021). The geometry of decision-making in individuals and collectives. *Proceedings of the National Academy of Sciences*, 118(50):e2102157118.